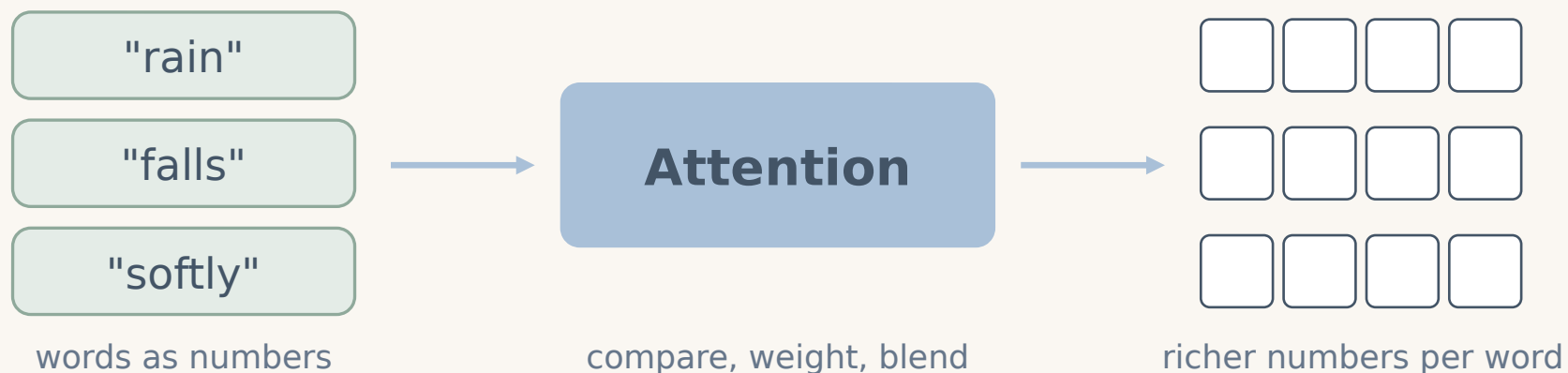


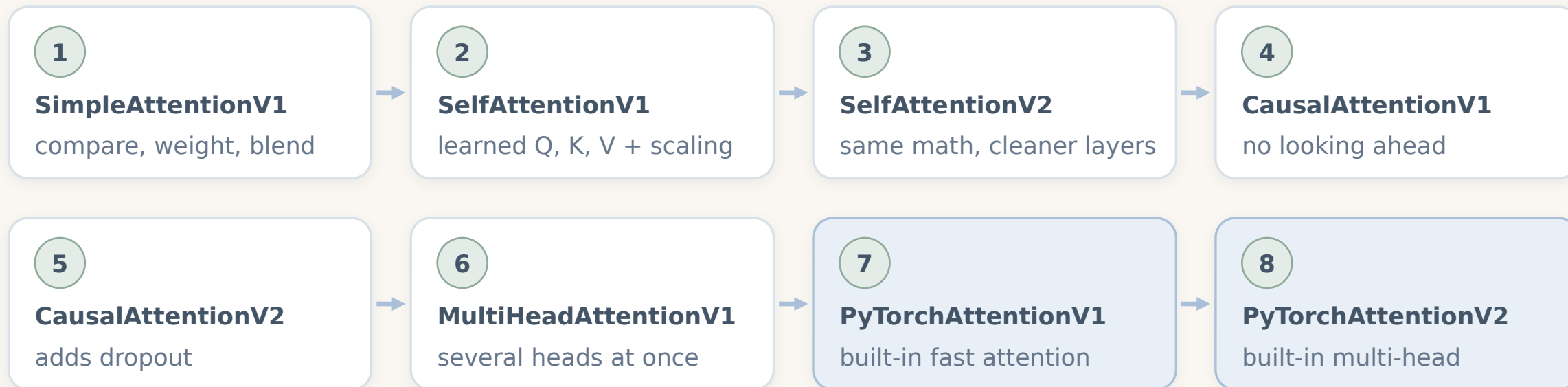
Your Attention Models

From "compare, weight, blend" to GPT-style multi-head attention



The journey: eight models

Each version keeps what came before and adds one new idea.



NEW WORDS

Model

here, a piece of code that computes attention

V1, V2

version 1, version 2 of the same idea

PyTorch

a Python library for building neural networks

Blue cards

versions PyTorch provides ready-made



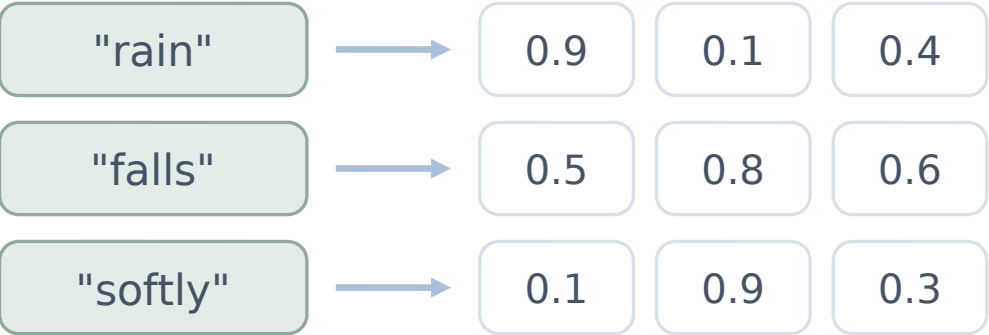
KEY IDEA

Learn the idea first, then each add-on, then the fast built-ins.

Starting point: words as numbers

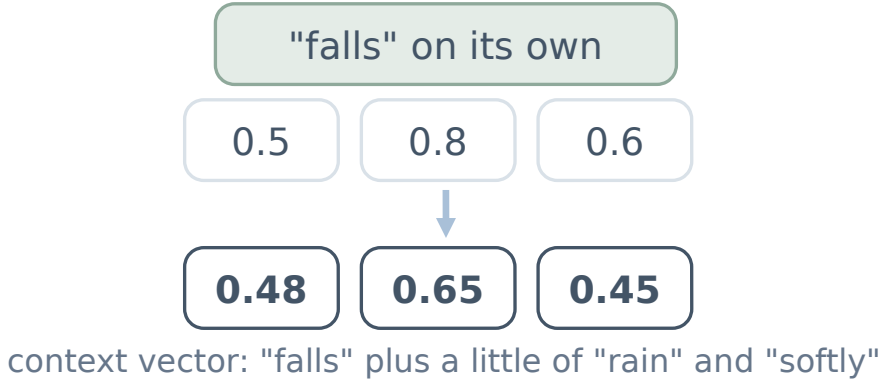
A model cannot read words, so every token is turned into a list of numbers.

Embeddings



Example numbers, 3 per token. Real models use hundreds.

What attention adds



NEW WORDS

Token

a small piece of text, often a word

Vector

a list of numbers

Embedding

the vector that stands for one token

Context vector

a token's vector after mixing in the other tokens



KEY IDEA

Attention turns each embedding into a context vector.

Tool 1: the dot product

A dot product compares two vectors and gives back one number: a score for how alike they are.

"falls"	0.5	0.8	0.6
× "softly"	0.1	0.9	0.3
multiply pairs	0.05	0.72	0.18

add them up: $0.05 + 0.72 + 0.18 =$ **0.95**

More alike, bigger score

"falls" with "falls"
same token **1.25**

"falls" with "softly"
fairly alike **0.95**

"rain" with "softly"
less alike **0.30**

NEW WORDS

Dot product

multiply matching numbers, then add them up

Score

the single number a dot product gives back

Alike

vectors whose numbers rise and fall together

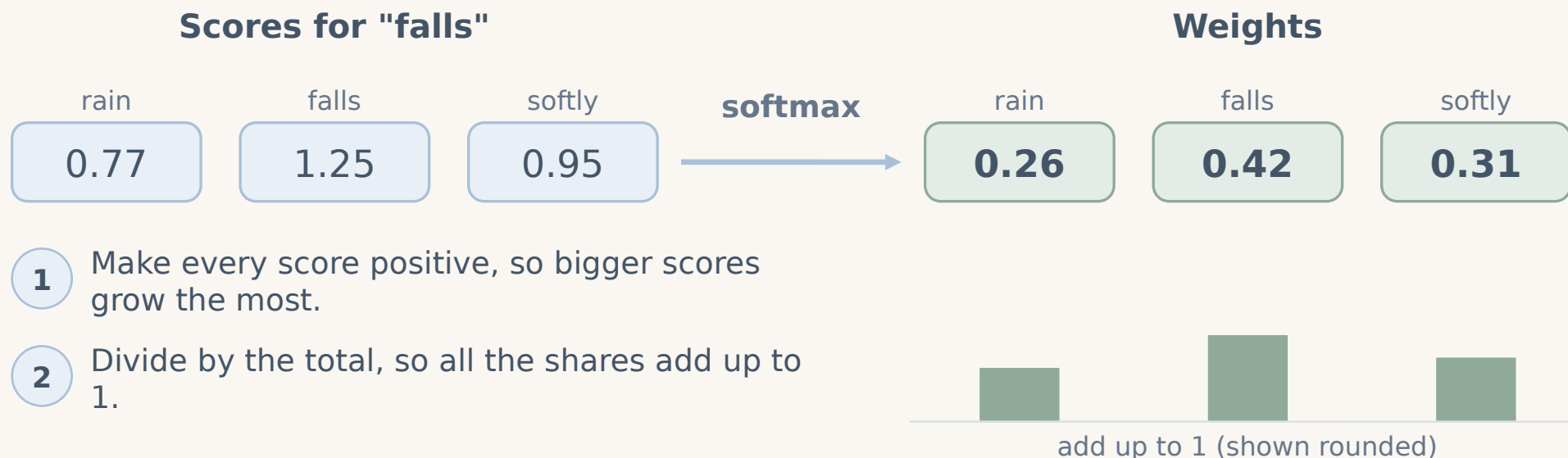


KEY IDEA

Bigger dot product = more alike.

Tool 2: softmax

Softmax turns a list of scores into weights: positive shares that add up to exactly 1.



NEW WORDS

Softmax

turns scores into shares that add up to 1

Weight

one share; 0.42 means 42% of the mix

Row

one token's list of scores or weights

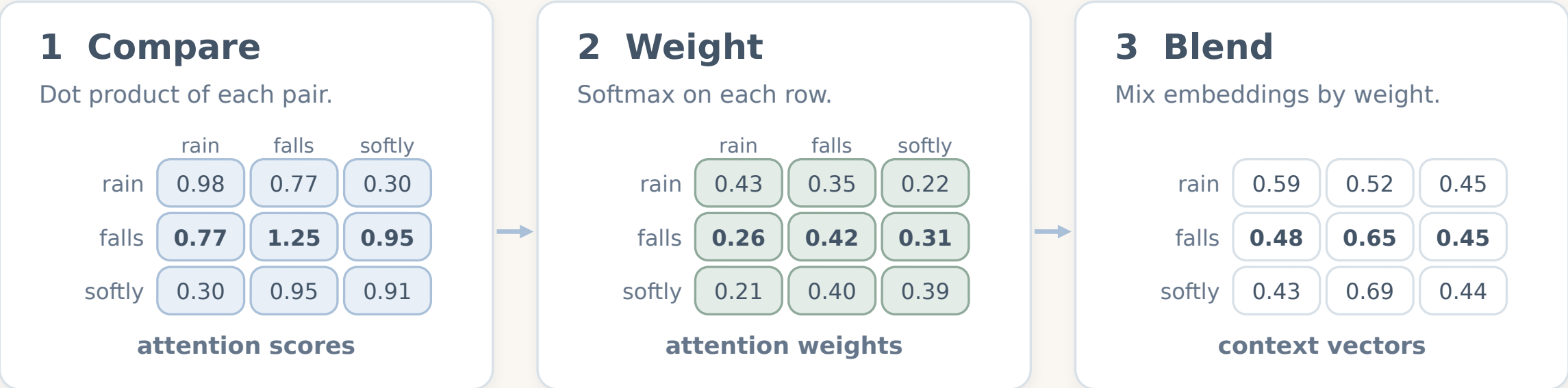


KEY IDEA

Bigger score, bigger share. The shares always add up to 1.

Simple attention: compare, weight, blend

SimpleAttentionV1 does all three steps for every token at once.



NEW WORDS	Attention score a dot product between two tokens	Attention weight a score after softmax	Matrix a grid of numbers; here one row per token
------------------	--	--	--

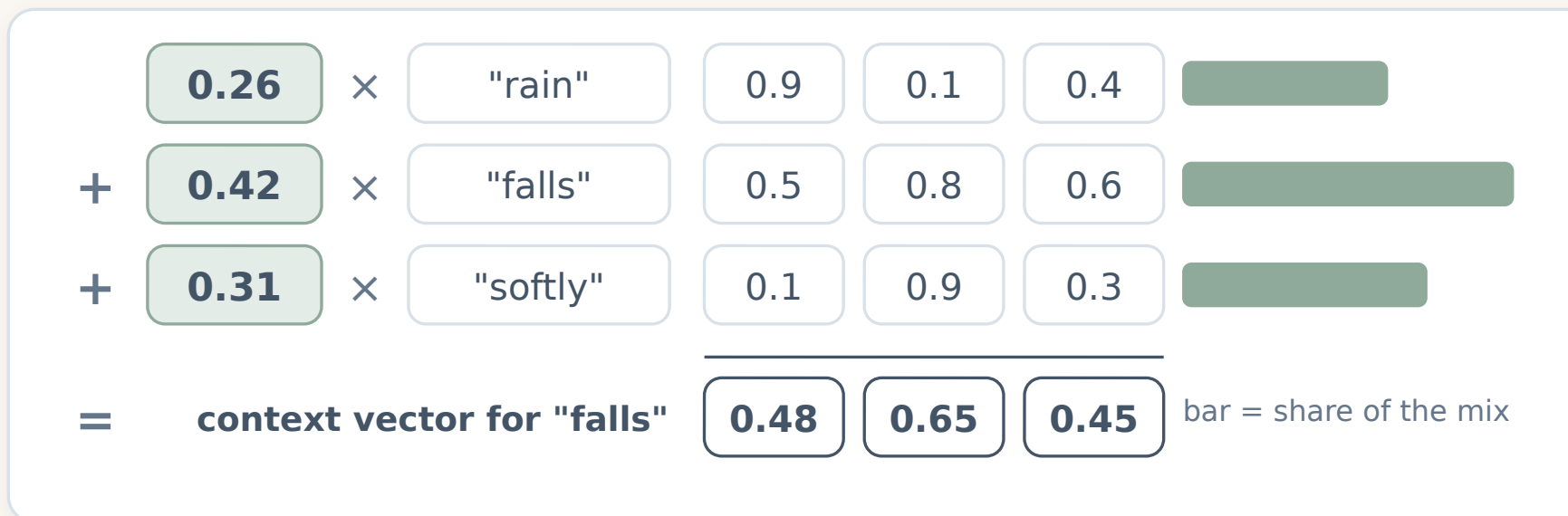


KEY IDEA

Compare, weight, blend: that is the whole idea of attention.

Zoom in: blending for "falls"

The "falls" row of weights says how much of each token to mix in.



NEW WORDS

Blend (weighted mix)

multiply each vector by its weight, then add them up

Context vector

the result: one new vector per token

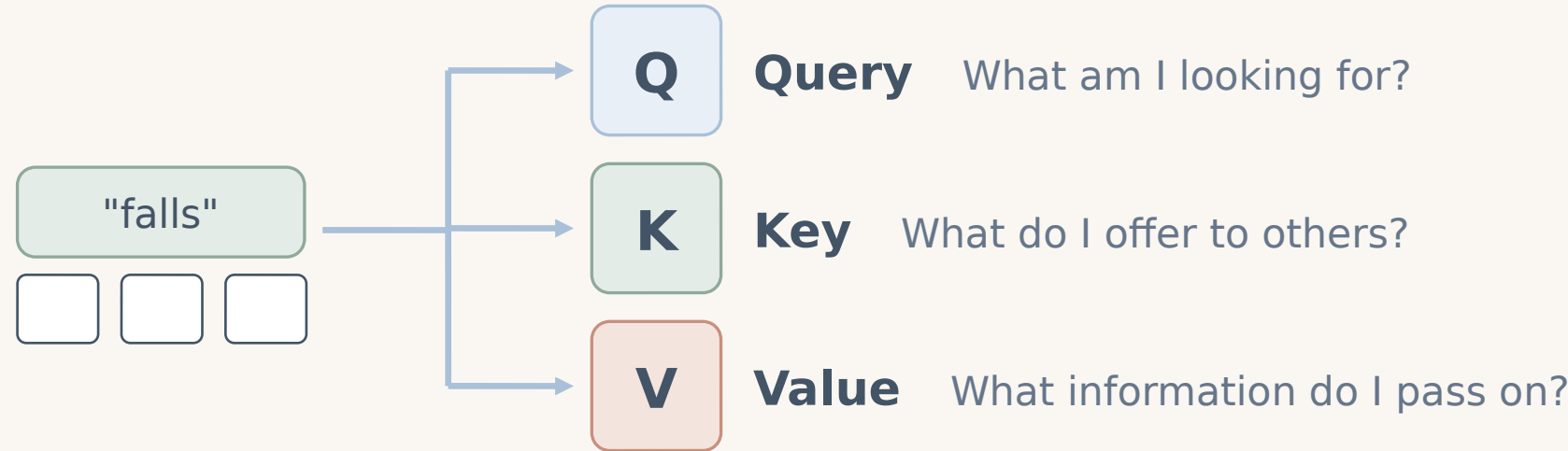


KEY IDEA

Tokens with bigger weights contribute more to the blend.

Query, Key, Value

Simple attention uses one vector for every job. Learned attention makes three vectors per token.



Like a library: your search (query) is matched with book labels (keys), then you read the matching books (values).

NEW WORDS

Learned matrix
a grid of numbers the model adjusts while training

Training
showing many examples so the model improves its numbers

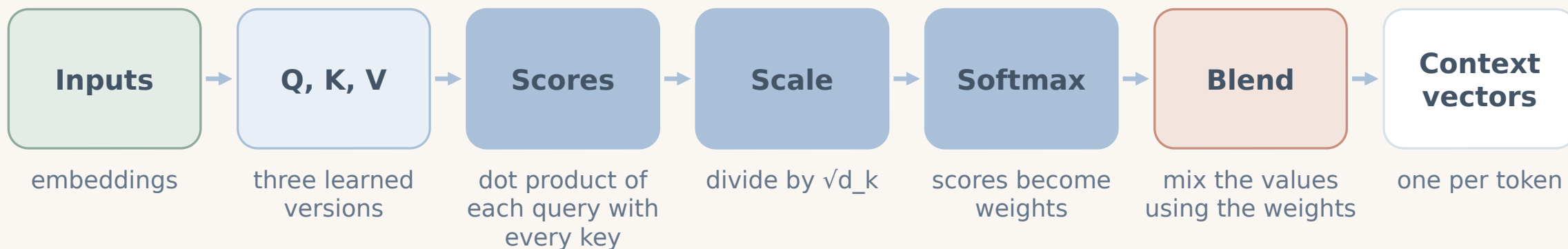
Self-attention
tokens in one sentence attend to each other



KEY IDEA
Queries and keys decide who to look at. Values are what gets blended.

Learned self-attention, step by step

SelfAttentionV1 and SelfAttentionV2 both follow this path.



NEW WORDS

d_k

how many numbers are in each key vector (its length)

$\sqrt{\text{ (square root) }}$

$\sqrt{16} = 4$, because $4 \times 4 = 16$

Scale

shrink every score by dividing by the same number



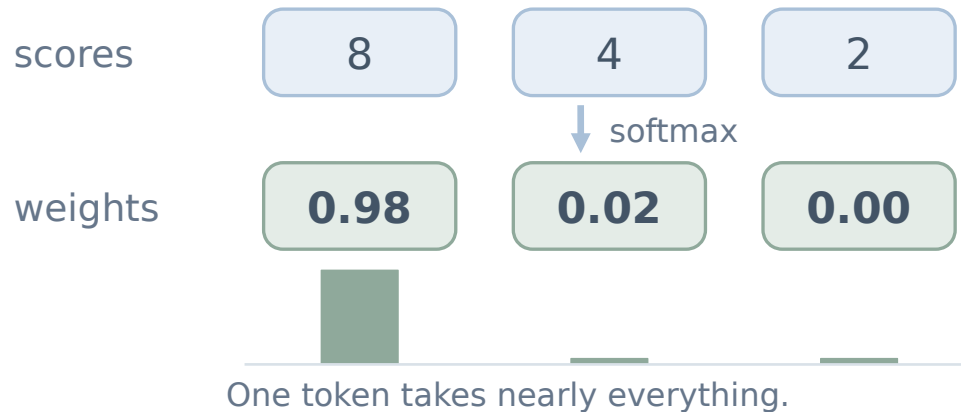
KEY IDEA

Same compare, weight, blend, now with learned Q, K, V and a scaling step.

Why divide by $\sqrt{d_k}$?

Longer vectors give bigger scores, and big scores make softmax put almost all the weight on one token.

Without scaling



With scaling: $\div 4$, because $\sqrt{16} = 4$



NEW WORDS

Overly confident

softmax gives nearly all the weight to one token and ignores the rest

Here $d_k = 16$

each key vector has 16 numbers, so we divide by $\sqrt{16} = 4$



KEY IDEA

Scaling keeps softmax from becoming overly confident.

Version 2: same math, cleaner code

SelfAttentionV2 does exactly the same calculation as SelfAttentionV1.

SelfAttentionV1

- Three raw learned matrices (nn.Parameter)
- Multiplies the inputs by them by hand

=

same
result

SelfAttentionV2

- Three nn.Linear layers
- Optional bias (the qkv_bias setting)
- The standard PyTorch building block

NEW WORDS

nn.Parameter

a PyTorch tensor the model is allowed to learn

nn.Linear

a ready-made layer: multiply by a learned matrix

Bias

an extra learned number added to each result

Tensor

PyTorch's name for a vector or grid of numbers

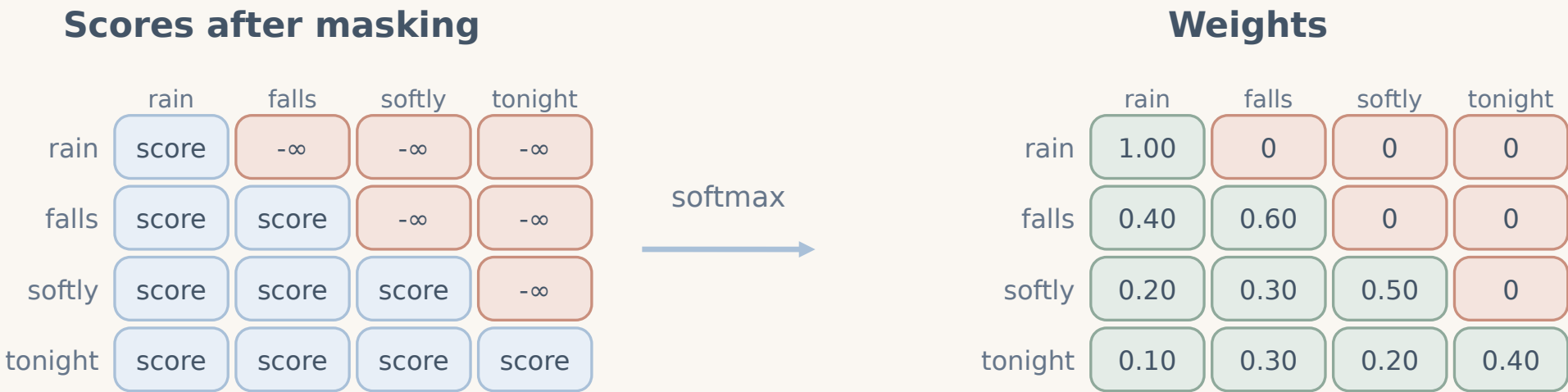


KEY IDEA

V2 is a cleaner implementation, not a different algorithm.

Causal masking: no peeking ahead

GPT learns by guessing the next token, so a token must not see the tokens after it.

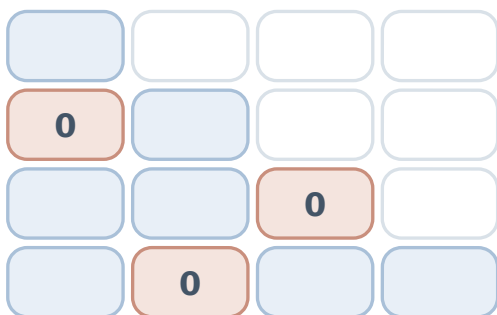


Dropout: learning with links removed

CausalAttentionV2 randomly drops some attention weights, but only while training.

While training

a few allowed links dropped at random



kept



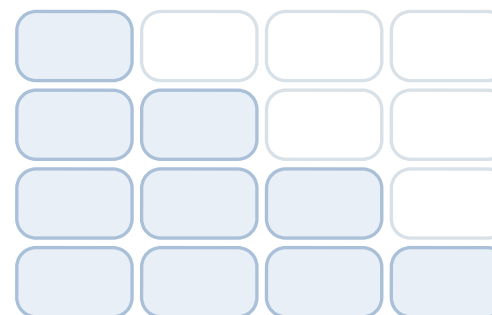
dropped



future (masked)

While evaluating

dropout is switched off



NEW WORDS

Dropout

randomly setting some weights to 0 while training

Evaluating

using the model after training; nothing is dropped

Overfitting

memorizing the examples instead of learning patterns

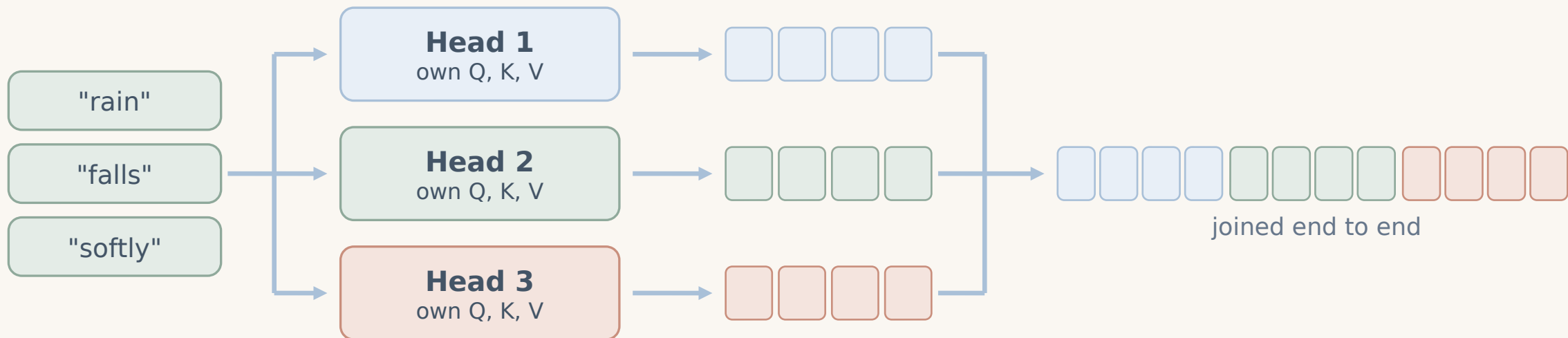


KEY IDEA

Dropout makes learning more robust, and it only runs during training.

Multiple heads, multiple perspectives

MultiHeadAttentionV1 runs several causal attention heads on the same input and joins their outputs.



output width = output_dimension × num_heads (here 4 × 3 = 12)

NEW WORDS

Head

one full attention unit with its own Q, K, V

num_heads

how many heads run side by side

output_dimension

numbers in each head's output vector

Concatenate

join vectors end to end



KEY IDEA

Several heads let the model notice several kinds of relationships at once.

The fast versions PyTorch provides

Same ideas, already written and optimized by PyTorch.

PyTorchAttentionV1

`scaled_dot_product_attention`

- Uses the input directly as Q, K and V
- Nothing learned: simple attention plus scaling
- Optional causal mask (`is_causal`)
- May use a fused kernel for speed

PyTorchAttentionV2

`nn.MultiheadAttention`

- Learned Q, K, V layers, split into heads
- Joins heads with a final output layer
- Input must be a batch
- Output keeps the input's width

NEW WORDS

Batch

several sentences processed together

Fused kernel

one combined, speed-tuned routine for the chip

`embedding_dim`

numbers per token; divisible by `num_heads`

Built-in

provided by PyTorch, no need to write it



KEY IDEA

In real projects, use the built-ins: same ideas, faster code.

Side by side

Filled dot = yes. "opt" = optional. Dash = no.

	Learned Q/K/V	Scaled	Causal mask	Dropout	Heads
SimpleAttentionV1	—	—	—	—	1
SelfAttentionV1	●	●	—	—	1
SelfAttentionV2	●	●	—	—	1
CausalAttentionV1	●	●	●	—	1
CausalAttentionV2	●	●	●	●	1
MultiHeadAttentionV1	●	●	●	●	many
PyTorchAttentionV1	—	●	opt	—	1
PyTorchAttentionV2	●	●	opt	—	many



KEY IDEA

GPT-style attention uses every idea in this table.

Glossary

Every term from this deck in one place.

Token	a small piece of text, often a word
Vector	a list of numbers
Embedding	the vector that stands for one token
Dot product	multiply matching numbers, then add; measures how alike
Attention score	dot product between two tokens
Softmax	turns scores into shares that add up to 1
Attention weight	a score after softmax
Context vector	a token's vector after blending in other tokens

Query	what a token is looking for
Key	what a token offers to others
Value	the information a token passes on
d_k	length of each key vector
Scaling	dividing scores by $\sqrt{d_k}$
Causal mask	blocks future tokens
Dropout	randomly zeroes some weights while training
Head	one attention unit with its own Q, K, V



KEY IDEA

Come back to this page whenever a word feels unfamiliar.

Recommended learning order

Work through the files in three stages.

1 The idea

SimpleAttentionV1

SelfAttentionV1

Watch: scores, weights, Q/K/V, scaling

2 Build it up

SelfAttentionV2

CausalAttentionV1

CausalAttentionV2

MultiHeadAttentionV1

Watch: masking, dropout, joining heads

3 Fast versions

PyTorchAttentionV1

PyTorchAttentionV2

Watch: which options match your own code



TAKEAWAY

Attention is compare, weight, blend. Everything else refines when and how.